

Pilha

Prof. Wesley Nunes Gonçalves¹
wnunesgoncalves@gmail.com

¹Universidade Federal de Mato Grosso do Sul

13 de novembro de 2024

1 Pilha

1 Pilha

Pilha

- Existem duas operações básicas:
 - Empilhar um novo elemento, inserindo-o no topo.
 - Desempilhar um elemento, removendo-o do topo.
- É comum nos referirmos a essas duas operações pelos termos em inglês push (empilhar) e pop (desempilhar).

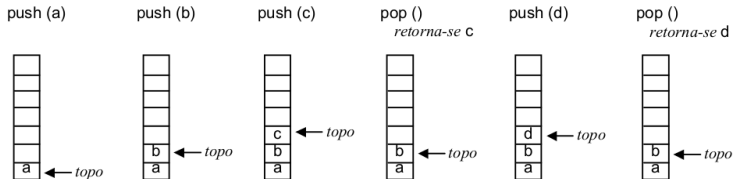


Figura: Push e Pop na pilha.

Pilha

- O nó da lista pode ser representado pela estrutura abaixo

```
1 struct no {
2     float info;
3     struct no* prox;
4 };
5 typedef struct no No;
6
7 struct pilha {
8     No* inicio;
9     int n;
10 };
11 typedef struct pilha Pilha;
```

- Inicializa

```
No* inicio
int n
```

- Inicializa

```

No* inicio → NULL
int n = 0

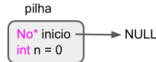
```


- Insere (push)

```

1 void push(Pilha* pilha, float v) {
2     No* p = (No*) malloc(sizeof(No));
3     p->info = v;
4     p->prox = pilha->inicio;
5     pilha->inicio = p;
6     pilha->n++;
7 }
8

```

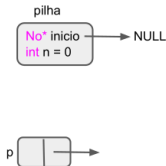


- Insere (push)

```

1 void push(Pilha* pilha, float v) {
2     No* p = (No*) malloc(sizeof(No));
3     p->info = v;
4     p->prox = pilha->inicio;
5     pilha->inicio = p;
6     pilha->n++;
7 }

```

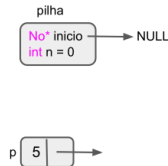


- Insere (push)

```

1 void push(Pilha* pilha, float v) {
2     No* p = (No*) malloc(sizeof(No));
3     p->info = v;
4     p->prox = pilha->inicio;
5     pilha->inicio = p;
6     pilha->n++;
7 }
8

```



Pilha

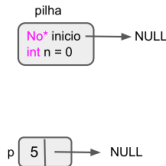
• Insere (push)

push(pilha, 5);

```

1 void push(Pilha* pilha, float v) {
2     No* p = (No*) malloc(sizeof(No));
3     p->info = v;
4     p->prox = pilha->inicio; ←
5     pilha->inicio = p;
6     pilha->n++;
7 }
8

```

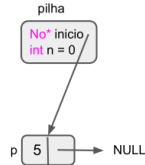


Pilha

● Insere (push)

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;  
5     pilha->inicio = p; ←  
6     pilha->n++;  
7 }  
8
```

push(pilha, 5);

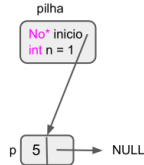


Pilha

● Insere (push)

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;  
5     pilha->inicio = p;  
6     pilha->n++;  
7 }  
8
```

push(pilha, 5);

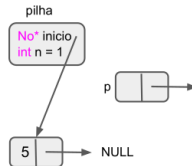


Pilha

● Insere (push)

push(pilha, 2);

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;  
5     pilha->inicio = p;  
6     pilha->n++;  
7 }  
8
```



Pilha

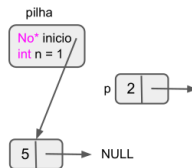
• Insere (push)

push(pilha, 2);

```

1 void push(Pilha* pilha, float v) {
2     No* p = (No*) malloc(sizeof(No));
3     p->info = v;
4     p->prox = pilha->inicio;
5     pilha->inicio = p;
6     pilha->n++;
7 }
8

```

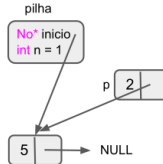


Pilha

● Insere (push)

push(pilha, 2);

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;   
5     pilha->inicio = p;  
6     pilha->n++;  
7 }  
8
```

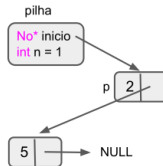


Pilha

● Insere (push)

push(pilha, 2);

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;  
5     pilha->inicio = p; ←  
6     pilha->n++;  
7 }  
8
```

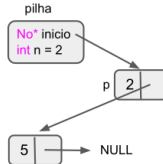


Pilha

● Insere (push)

push(pilha, 2);

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;  
5     pilha->inicio = p;  
6     pilha->n++;  
7 }  
8
```



Pilha

● Inserção (push)

```
1 void push(Pilha* pilha, float v) {  
2     No* p = (No*) malloc(sizeof(No));  
3     p->info = v;  
4     p->prox = pilha->inicio;  
5     pilha->inicio = p;  
6     pilha->n++;  
7 }
```

Pilha

• Remoção (pop)

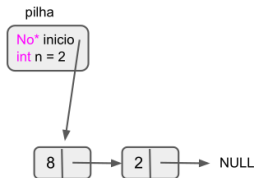
```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info; ←
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);

float v = 8



Pilha

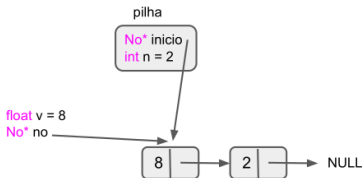
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio; ←
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);



Pilha

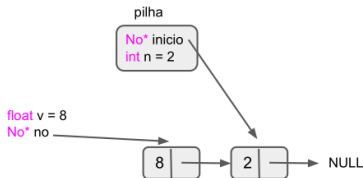
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox; ⇐
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);



Pilha

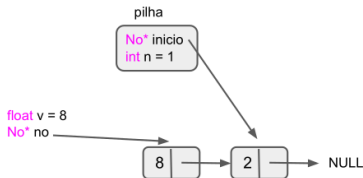
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);



Pilha

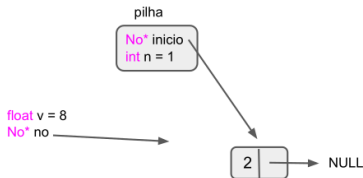
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no); ←
11    return v;
12 }
13

```

pop(pilha);



Pilha

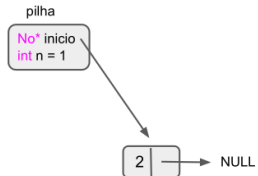
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf("Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no);
11    return v; ←
12 }
13

```

pop(pilha);



Pilha

• Remoção (pop)

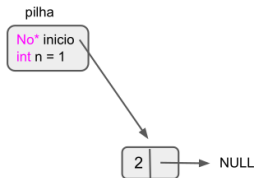
```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info; ←
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);

float v = 2



Pilha

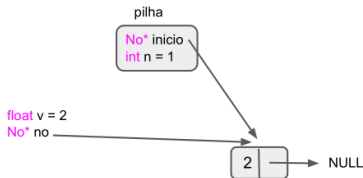
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio; ←
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);



Pilha

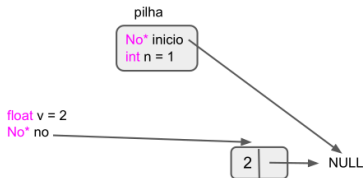
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox; ⇐
9     pilha->n--;
10    free(no);
11    return v;
12 }
13

```

pop(pilha);



Pilha

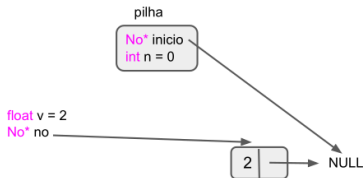
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--; ⇐
10    free(no);
11    return v;
12 }
13

```

pop(pilha);



Pilha

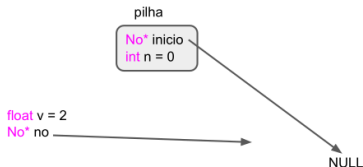
• Remoção (pop)

```

1 float pop(Pilha* pilha) {
2     if(pilha->inicio == NULL) {
3         printf(" Pilha vazia!");
4         return -1.0;
5     }
6     float v = pilha->inicio->info;
7     No* no = pilha->inicio;
8     pilha->inicio = pilha->inicio->prox;
9     pilha->n--;
10    free(no); ←
11    return v;
12 }
13

```

pop(pilha);

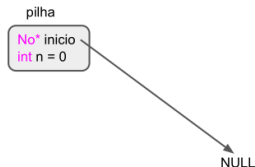


Pilha

• Remoção (pop)

```
1 float pop(Pilha* pilha) {  
2     if(pilha->inicio == NULL) {  
3         printf(" Pilha vazia!");  
4         return -1.0;  
5     }  
6     float v = pilha->inicio->info;  
7     No* no = pilha->inicio;  
8     pilha->inicio = pilha->inicio->prox;  
9     pilha->n--;  
10    free(no);  
11    return v; ←  
12 }  
13
```

pop(pilha);



Pilha

● Remoção (pop)

```
1 float pop(Pilha* pilha) {  
2     if(pilha->inicio == NULL) {  
3         printf("Pilha vazia!");  
4         return -1.0;  
5     }  
6     float v = pilha->inicio->info;  
7     No* no = pilha->inicio;  
8     pilha->inicio = pilha->inicio->prox;  
9     pilha->n--;  
10    free(no);  
11    return v;  
12 }  
13
```

Pilha

- Limpar pilha

```
1 void libera(Pilha* pilha) {  
2     No* p = pilha->inicio;  
3     while(p != NULL) {  
4         No* t = p->prox;  
5         free(p);  
6         p = t;  
7     }  
8     free(pilha);  
9 }  
10
```

Pilha

- Imprimir pilha (importante para debug)

```
1 void imprime(Pilha* pilha) {  
2     No* p = pilha->inicio;  
3     while(p != NULL) {  
4         printf("%f, ", p->info);  
5         p = p->prox;  
6     }  
7     printf("\n");  
8 }  
9
```

Pilha

● Main

```
1 int main() {  
2     Pilha* pilha = inicializa();  
3     push(pilha, 10);  
4     push(pilha, 20);  
5     push(pilha, 30);  
6     imprime(pilha);  
7     pop(pilha);  
8     imprime(pilha);  
9     push(pilha, 40);  
10    imprime(pilha);  
11    pop(pilha);  
12    imprime(pilha);  
13    libera(pilha);  
14    return 0;  
15 }  
16
```

Sumário

1 Pilha